

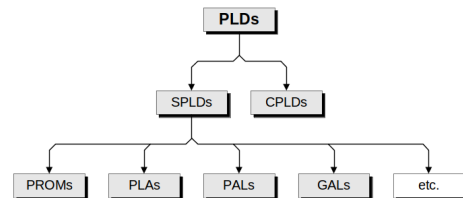
ISP - Opracowanie Gumisiowe

Autorstwa M., M., M., i M.

Część A

1. Wymień trzy kategorie układów programowalnych (podział pod względem złożoności architektury) i scharakteryzuj krótko każdą z nich.

Podział układów programowalnych (bez FPGA)



- SPLD – Simple Programmable Logic Devices (do 500 bramek, max. 24 makrokomórki, max. 40 linii I/O)
- CPLD – Complex Programmable Logic Devices
- PROM – Programmable Read Only Memory
- PLA – Programmable Logic Array 1975 Signetics/Philips, układ PLS100. Duża cena, mała szybkość działania.
- PAL – Programmable Array Logic 1978 Monolithic Memories/AMD
- GAL – Generic Array Logic 1985 Lattice

"The Design Warrior's Guide to EPLDs, ISBN 0750606040, Copyright© 2004 Mentor Graphics Corp."

- **SPLD** - proste układy bazujące na programowalnych macierzach OR i/lub AND (np. pamięć PROM - macierz OR, PAL - macierz AND, PLA - macierz AND i OR, GAL - reprogramowalne układy PAL, z ozłiwością "pożyczania" termów z sąsiednich komórek)
- **CPLD** - składają się z bloków typu SPLD połączonych dodatkową wewnętrzną programowalną magistralą połączeniową
- **FPGA** - najbardziej zaawansowane - składają się z wielu bloków logicznych połączonych konfigurowalną siecią

2. Jakiej kategorii układów programowalnych dotyczy podział na: coarse grade, medium grade i fine grade. Scharakteryzuj krótko cechy wyżej wymienionych rodzajów układów.

FPGA; nie jestem super pewien czy o to chodzi, w prezentacji to się inaczej nazywa i nie jest opisane

- **coarse grade** - Duże bloki funkcjonalne (np. ALU, rejestry), wysoka wydajność, mniejsza elastyczność, lepsze do specyficznych zastosowań.
- **medium grade** - Średniej wielkości bloki (np. małe funkcje logiczne), kompromis między elastycznością a wydajnością, bazują na multiplexerach.
- **fine grade** - Małe bloki logiczne (np. pojedyncze bramki), wysoka elastyczność ale mniej wydajne. Bazują na przerzutnikach i prostych 2-3 wejściowych układach.

nie jestem super pewien czy o to chodzi, w prezentacji to się inaczej nazywa i nie jest opisane...

3. Jaki rodzaj pamięci konfiguracji dominuje w układach CPLD, a jaki w FPGA.

CPLD - (E)PROM, EEPROM, FLASH

FPGA - SRAM (ładowane z FLASH przy starcie)

tbh to jest mocno na "vibes" zrobione w prezentacji nie ma tego explicitly, więc kidna bazuje to na pól słówkach i enigmatycznych obrazkach. internet mi mówi że ogólnie to jest różnie Xilinx podobno robi SRAM FPGA ale inni robią takie z FLASH więc chuj wie

4. Wymień rodzaje pamięci konfiguracji – podaj ich wady i zalety.

Rodzaj	Zalety	Wady
PROM (Fuse/Antifuse)	- zajmuje mało miejsca w układzie - trwała (nie kasuje się po utracie zasilania) - małe opory	nie reprogramowalna
EPROM	- reprogramowalna - trwała	Nie wygodna w użyciu
EEPROM	trwała (nie kasuje się po utracie zasilania)	Wolniejszy zapis niż FLASH
FLASH	- Szybszy zapis niż EPROM - trwała (nie kasuje się po utracie zasilania) - Działanie od razu po starcie układu	- Wolniejsza niż SRAM - Większe zużycie energii
SRAM	Szybsze programowanie	- duża powierzchnia w układzie - ulotna (kasuje się po utracie zasilania)

5. W układzie Spartan3E w jednej komórce LUT można zrealizować pewne funkcje logiki. Podaj ile maksymalnie zmiennych wejściowych może mieć dowolna funkcja logiczna zrealizowana w jednej komórce LUT, ile maksymalnie zmiennych wejściowych może mieć dowolna funkcja logiczna zrealizowana za pomocą jednego Slice oraz ile maksymalnie wejść mogą mieć pewne funkcje logiczne wykonane w oparciu o jeden Slice.

- LUT - 4 wejścia
- SLICE (all) - 5 wejść
- SLICE (some) - 9 wejść

Mux	Usage	Input Source	Total Number of Inputs per Function		
			For Any Function	For Mux	For Limited Functions
F5MUX	F5MUX	LUTs	5	6 (4:1 mux)	9
F1MUX	F6MUX	F5MUX	6	11 (8:1 mux)	19
	F7MUX	F6MUX	7	20 (16:1 mux)	39
	F8MUX	F7MUX	8	37 (32:1 mux)	79

6. Wymień parametry końcówki I/O układu Spartan3E które można konfigurować.

Wzięte z [opracowania IUP 2015](#):
w prezentacji ni chuja tego nie ma

- standard napięciowy TTL/CMOS (kilka różnych odmian) - określa poziomy napięć stanów wysokich i niskich dla komórek wejściowych jak i wyjściowych. Trzeba na to zwracać uwagę gdyż nie wszystkie standardy można łączyć ze sobą (np. wyjść TTL z wejściami niskonapięciowymi CMOS). Złe dobranie standardów może spowodować uszkodzenie układów albo problem z identyfikacji stanu logicznego (gdy np. poziom stanu jednego standardu trafia akurat w zakres napięć nieustalonych drugiego standardu).
- programowalny max prąd wyjściowy
- programowalny pull-up/pull-down określa do jakiego potencjału ma być dołączony pin wejściowy.
- „clamping” - do pad'u mogą być dołączone diody stanowiące zabezpieczający ogranicznik napięcia (3.3V), stosowane do aplikacji PCI. Włączenie tych diod i połączenie układu do jakiegoś wyjścia TTL

(+5V) może spowodować przepływ znacznego prądu przez diody i uszkodzić układ. Diody te są domyślnie wyłączone.

7. Wymień tryby konfiguracji układu Spartan3E.

- Master Serial
- SPI
- BPI
- Slave Parallel
- Slave Serial
- JTAG

8. Opisz właściwości i zastosowanie bloku „week-keeper” w układzie Spartan3E.

Układ podtrzymujący wartość sygnału na magistrali, gdy wszystkie sterowniki są wyłączone. Działa na zasadzie słabego podciągania (pull-up) lub ściągania (pull-down) sygnału za pomocą rezystora, utrzymując ostatni stan logiczny linii. Zapobiega fluktuacji napięcia utrzymując stan na ostatnim stanie użytecznym zapobiegając występowaniu stanów pośrednich i zwiększaniu zużycia mocy.

Source: <https://adaptivesupport.amd.com/s/article/19826>

9. Wyjaśnij pojęcie „local routing”.

Połączenia między elementami logicznymi znajdującymi się blisko siebie, np. w tym samym lub sąsiednim bloku logicznym. Służy do przesyłania sygnałów na krótkie dystanse. Połączenia takie mają mniejsze opóźnienia.

10. Wyjaśnij pojęcie „general purpose routing”.

Uniwersalna sieć połączeń, która łączy dowolne elementy układu FPGA, także te oddalone od siebie. Umożliwia swobodną komunikację między blokami logicznymi. Bazuje na Switch Blocks - programowalnych interkonektach. Połączenia takie mają większe opóźnienia. Możemy je podzielić na połączenia o krótkim dystansie, średnim dystansie, na cały układ.

Source: <http://www.tutorial-reports.com/computer-science/fpga/routing.php>

Część B

11. Wyjaśnij pojęcie „global routing”.

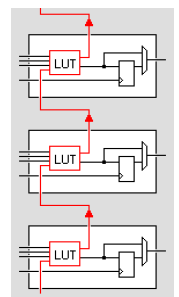
Specjalna sieć służąca do rozprowadzania sygnałów globalnych, takich jak zegar (clock), sygnały resetu, enable, itp., do wielu elementów w całym układzie FPGA.

- Musi zapewniać odpowiednie własności sygnałów, np. niskie opóźnienia i wyrównanie czasowe dla sygnałów zegarowych.

12. Wyjaśnij pojęcie „dedicated routing”.

Dedykowane połączenia dla pewnej funkcjonalności np. budowy sumatorów - sygnał carry musi być bardzo szybko przekazany do następnego bloku - realizuje się takie połączenie w sposób bezpośredni między blokami sumatora.

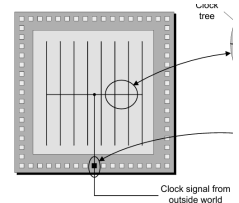
- niskie opóźnienia
- wysoka szybkość
- brak potrzeby wykorzystania dodatkowej logiki



13. Wyjaśnij pojęcie „clock skew” oraz podaj sposoby jego zmniejszania lub kompensacji.

Clock skew występuje kiedy z jednego źródła sygnału zegarowego, sygnał dociera do różnych bloków/komponentów w różnym czasie. Przesunięcie fazowe sygnałów zegarowych względem siebie utrudnia przekazywanie danych pomiędzy domenami zegarowymi,

Kompensację tego zjawiska można uzyskać wykorzystując drzewo dystrybucji sygnału zegarowego wewnątrz chipu, które zajmuje się dystrybucją zegara o określonej fazie do przerzutników stosując odpowiednie opóźnienia.

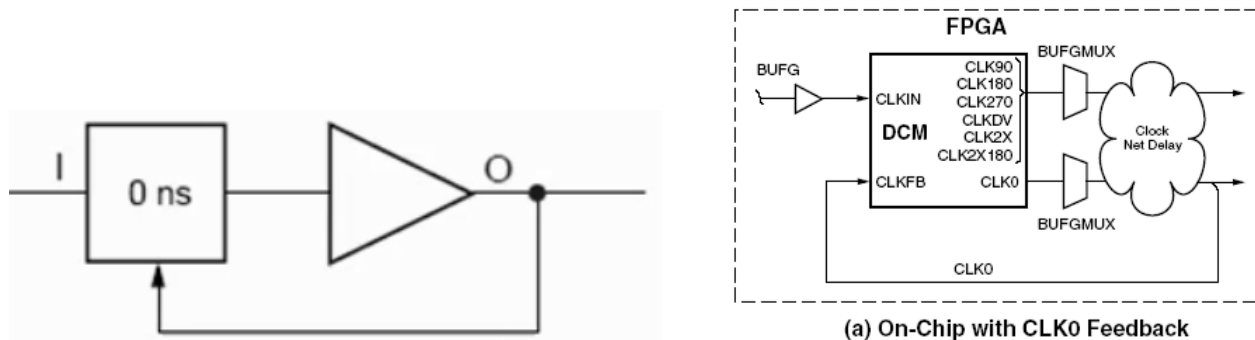


Wykorzystuje się także układ Clock Managera, który sprawia, że sygnał wyjściowy jest zgodny z fazą sygnału dystrybuowanego przez drzewo opisane powyżej. Może dostosować fazę sygnału wyjściowego do danej domeny zegarowej.

14. Wyjaśnij pojęcie Delay Locked Loop, narysuj schemat takiej pętli i podaj zastosowania.

DLL dostosowuje fazę sygnału zegara poprzez wprowadzenie zmiennego opóźnienia, tak aby wyrównać go z odniesieniem (np. wejściowym zegarem lub sygnałem danych).

Dąży do tego, aby opóźnienie sygnału w bloku kwadratowym na rysunku poniżej było takie, aby różnica w fazie wchodzących sygnałów była równa 0ns.



DLL Jest stosowana np. w układach FPGA w Device Clock Managerze, w celu zapewnienia, że sygnał zegarowy w sieci dystrybucji będzie miał dokładnie taką fazę jak sygnał zegarowy na pinie wejściowym. Innym zastosowaniem jest synchronizacja danych z zegarem.

15. Jakie sygnały generuje układ DCM w układzie Spartan3E. Jakie są wymagania odnośnie parametrów sygnału zegarowego.

Sygnały przesunięte w fazie o odpowiednio 0, 90, 180, 270 stopni, ewentualnie pomnożone/podzielone przez 2 albo programowalną wartość:

- CLK0, CLK90, CLK180, CLK270
- CLK2X, CLK2X180
- CLKFX, CLKFX180
- CLKDV

Informacja czy nastąpiła synchronizacja w pętli DLL:

- LOCKED

Główne wymagania odnośnie parametrów wyjściowego sygnału zegarowego względem wejściowego to:

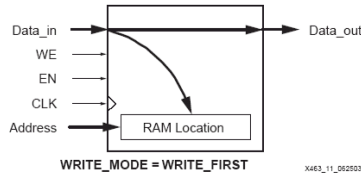
- równy współczynnik wypełnienia sygnału zegarowego (czas trwania '1' i '0' wynosi po 50% cyklu),
- redukcja clock skew/synchronizacja faz sygnałów/(minimalizacja różnicy w fazie sygnałów),
- ??? dostęp (osobne sygnały) do przesunięcia zegara w fazie,

- ??? dostę (osobne sygnały) do zsyntezowanego sygnału zegarowego o częstotliwości zgodnej z zaprogramowanymi wartościami dzielnika i mnożnika.

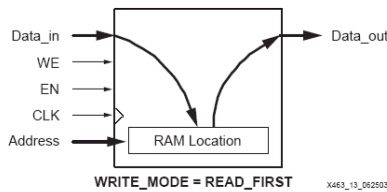
16. Wymień i scharakteryzuj tryby zapisu jednoportowych pamięci Block-RAM w układach Spartan3E.

Trzy tryby zapisu - w każdym wartość zostaje zapisana do pamięci:

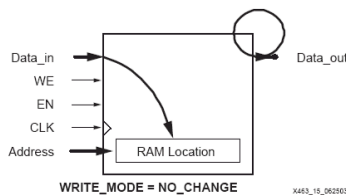
- WRITE_FIRST - oprócz zapisu, w tym samym cyklu zegarowym, obecna wartość zostaje przekierowana na wyjście,



- READ_FIRST - oprócz zapisu, poprzednia zawartość pamięci zostaje wystawiona na wyjście,



- NO_CHANGE - oprócz zapisu, nic więcej (dane na zatraskach wyjściowych nie ulegają zmianie podczas zapisu).



17. Wymień możliwe zastosowania jednoportowych pamięci Block-RAM w układach Spartan3E.

Zastosowania:

- Duże pamięci synchroniczne SRAM jednoportowe z możliwością inicjacji zawartości podczas konfiguracji,
- Duże pamięci ROM,
- Synchroniczne i asynchroniczne FIFO,
- Pamięć typu LIFO (stos) do procesorów,
- Szybka implementacja złożonych funkcji logicznych,
- Rejestry przesuwne,
- Złożone maszyny stanów,
- Szybkie liczniki,
- Bufory cyrkulacyjne,
- Linie opóźniające (np. do FIR).

18. Wymień właściwości i zastosowania 2-portowych pamięci Block-RAM w układach Spartan3E.

Właściwości:

- możliwe jednoczesny(e) zapis(y)/odczyt(y) w tym samym czasie (cyklu zegara) - współbieżny wielodostęp
- Z powodu jednoczesnego dostępu mogą wystąpić kolizje,
- każdy z portów posiada osobny zegar - może więc pełnić rolę interfejsu do pamięci RAM z różnych domen zegarowych (układów taktowanych zegarami o różnych częstotliwościach).

Zastosowania:

- Duże pamięci synchroniczne SRAM dwuportowe z możliwością inicjacji zawartości podczas konfiguracji,

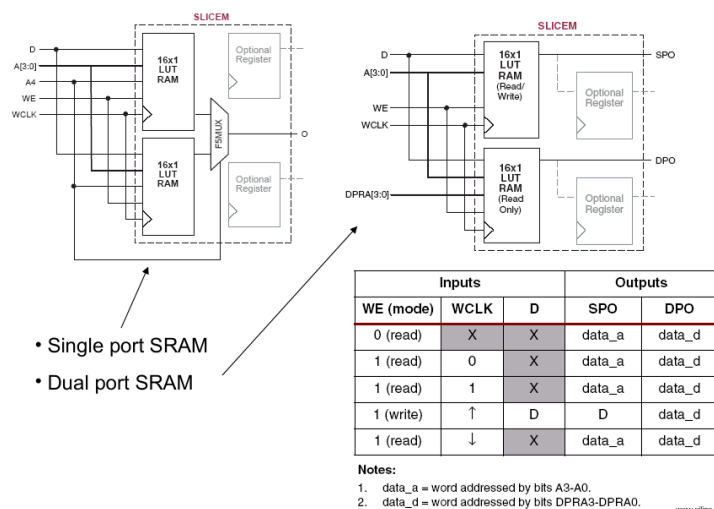
- Dostęp do pamięci z różnych domen zegarowych,
- Duże pamięci ROM,
- Synchroniczne i asynchroniczne FIFO,
- Pamięć typu LIFO (stos) do procesorów,
- Szybka implementacja złożonych funkcji logicznych,
- Rejestry przesuwne,
- Złożone maszyny stanów,
- Szybkie liczniki,
- Bufory cyrkulacyjne,
- Linie opóźniające (np. do FIR).

19. Wymień funkcje które może w komórce CLB układu Spartan3E spełniać blok LUT.

Funkcje LUT:

- Dowolna 4-wejściowa funkcja logiczna
- jednoportowy blok pamięci RAM adresowany 4 bitami
- Dynamiczny 16-bitowy rejestr przesuwny z szeregowym I/O, bez resetu - tylko z zegarem,
- układ logiczny z nawet 9 wejściami - chyba chodzi o to że tu jest 9 wejść:

Spartan-3E (17) CLB – distributed RAM



- Single port SRAM
- Dual port SRAM

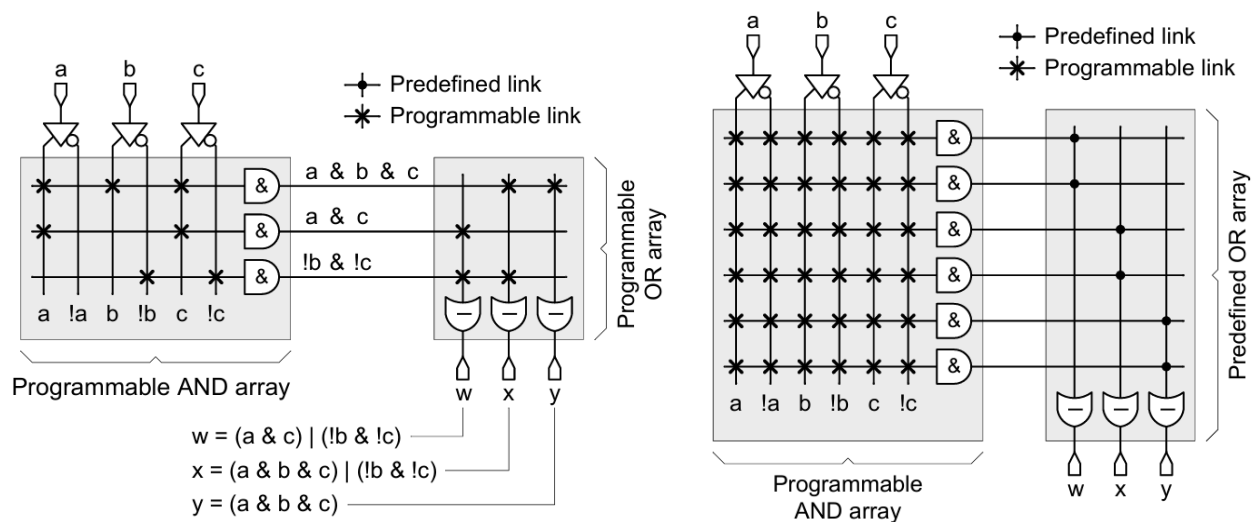
można o tym wspomnieć, ale nie jest to odpowiedź na pytanie:

- częściowo-dual (tylko odczyt) blok pamięci RAM adresowany 4 bitami w przypadku połączenia dwóch bloków LUT,
- jednoportowy blok pamięci RAM adresowany 5 bitami w przypadku połączenia dwóch bloków LUT za pomocą F5MUX),

20. Wymień różnice w budowie układów PLA i PAL.

PLA – Programmable Logic Array

PAL – Programmable Array Logic



W układach PLA (Programmable Logic Array) występują 2 matryce programowalne - matryca AND do tworzenia mintermów i matryca OR do ich sumowania. Z racji na programowalność obu matryc i wynikające z tego opóźnienia powstały układy PAL (Programmable Array Logic). W tych układach programowalna jest jedynie matryca AND, to ta do tworzenia mintermów, natomiast w matrycy OR połączenia są przypisane na stałe - pozwoliło to zmniejszyć opóźnienia.

Cześć C

21. Wymień i krótko scharakteryzuj poziomy abstrakcji stosowane w opisie sprzętu.

Idk czy o to chodzi?

Poziomy abstrakcji projektu

Poziom złożoności	Reprezentacja behawioralna:	Reprezentacja strukturalna:
System	Opis w języku naturalnym	Jednostka centralna, dysk, antena, ...
Układ scalony	Zestaw algorytmów (Behavioral Level)	Mikroprocesor, pamięć, układ I/O, ...
Rejestry	Przepływ danych (Register Transfer Level)	Rejestry, ALU, ROM, MUX, licznik, ...
Bramki	Równania logiczne (Data Flow)	Funktory logiczne: and, or, xor, ... (Gate Level)
Układ elektr.	Równania różniczkowe (Transistor Level)	Tranzystory, rezystory, kondensatory, ...
Layout, krzem	Fizyka półprzewodników	Geometria struktur, ...

22. Wymień metody opisu układów cyfrowych.

Metody opisu układów cyfrowych

- Schemat ideowy.
- Schemat blokowy.
- Diagram czasowy.
- Graf maszyny stanów.
- Tabela prawdy.
- Równania logiczne.

23. Co to jest podzbiór syntezywalny języka VHDL, podaj przykład instrukcji nie syntezywanej.

To część języka możliwa do implementacji w sprzęcie.

Przykład nie syntezywalny: ****** (potęgowanie), **mod**, **rem**, **real**

24. Co to jest funkcja rezolucji.

Mechanizm rozstrzygania konfliktów przy przypisaniach do tego samego sygnału. Działa tylko dla **std_logic** i **std_logic_vector**, umożliwia np. implementację logiki trójstanowej

Typy sygnałów – funkcja rezolucji

Funkcja rezolucji dla typu **std_logic**:

U	X	0	1	Z	W	L	H	-	
U	U	U	U	U	U	U	U	U	U
U	X	X	X	X	X	X	X	X	X
U	X	0	X	0	0	0	0	X	0
U	X	X	1	1	1	1	1	X	1
U	X	0	1	Z	W	L	H	X	Z
U	X	0	1	W	W	W	W	X	W
U	X	0	1	L	W	L	W	X	L
U	X	0	1	H	W	W	H	X	H
U	X	X	X	X	X	X	X	X	-

- **std_logic** oznacza typ sygnału. Jest to podstawowy typ opisujący sygnał logiczny. Może on przyjmować wartości **0** i **1** ale także:

Z stan wysokiej impedancji;

- don't care (wartość logiczna bez znaczenia, np. funkcja **std_match**);

Pozostałe wartości dotyczą tylko symulacji:

U niezainicjowany (domyślnie nadawany na początku symulacji);

X nieokreślony silny sygnał; **W** nieokreślony słaby sygnał;

L, **H** do opisu słabych sygnałów 0 i 1.

Funkcja rezolucji pozwala na syntezę logiki trójstanowej wewnątrz układu.

Syntezer nie zgodzi się na przypisanie do sygnału innej wartości niż 0, 1 lub

Z. Nie wszystkie układy programowalne posiadają wewnętrzne bramki

trójstanowe! Uwaga: sygnały ZUXLHW nie mogą być pisane małymi literami!

25. Podaj sposoby unikania stanów zabronionych w maszynach stanów.

- a. Stosowanie pełnych klauzul **else**
- b. inicjalizacja wszystkich stanów
- c. stosowanie **others**
- d. unikanie niepełnych przypisań

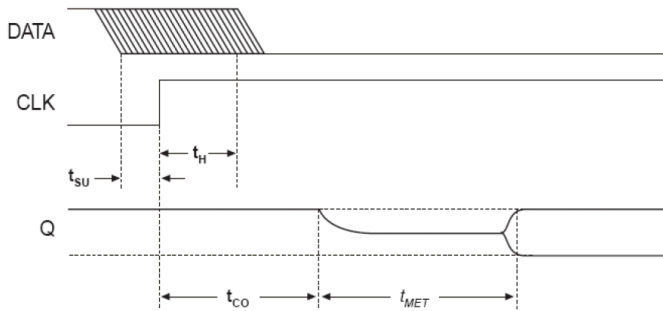
26. Co to jest metastabilność przerzutników

Zjawisko nieokreślonego stanu przerzutnika przy naruszeniu czasu ustawiania/trzymania – grozi błędami logicznymi..

Metastabilność (1)

Unikanie stanów zabronionych, c.d.:

Użycie potoku złożonego z dwóch lub więcej przerzutników synchronizujących może być konieczne jeżeli występuje duże prawdopodobieństwo pojawienia się zjawiska metastabilności w pierwszym przerzutniku synchronizującym. Zjawisko to polega na pozostawaniu przerzutnika przez pewien czas w równowadze chwiejnej pomiędzy dwoma stanami stabilnymi. Na rysunku czas ten zaznaczono jako t_{MET} .



-- przykład procesu realizującego
-- dwa przerzutniki synchronizujące

```
signal asynch_in : std_logic;
signal q : std_logic;
signal synch_out : std_logic;
```

```
synch: process (clk) is
begin
    if rising_edge(clk) then
        q <= asynch_in;
        synch_out <= q;
    end if;
end process;
```

Metastabilność (2)

$$MTBF = \frac{e^{(C_2 \times t_{MET})}}{C_1 \times f_{CLOCK} \times f_{DATA}}$$

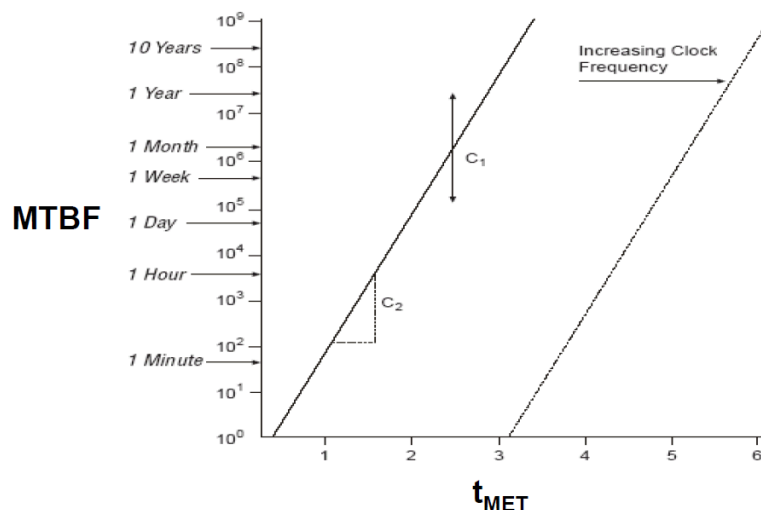
MTBF – Mean Time Between Failures

f_{data} – częstotliwość zmian w sygnale danych

f_{clock} – częstotliwość zegara

t_{MET} – dodatkowy czas zarezerwowany dla ustalenia się stanu wyjściowego

Stałe C_1 i C_2 zależą od właściwości przerzutników



Metastabilność (3)

```
-- przykład procesu realizującego dwa przerzutniki synchronizujące
-- oraz układ likwidacji drgań zestyków

signal key_in : std_logic;           -- sygnał bezpośrednio z przycisku
signal q : std_logic;
signal key_synch : std_logic;
signal key_stable_out : std_logic;   -- stabilny sygnał przycisku (out)
constant delay_const : integer := 10000; -- opóźnienie syg. stabilnego

synch: process (clk) is
variable delay_cntr : integer := 0;
begin
    if rising_edge(clk) then
        q <= key_in;
        key_synch <= q;
        if (key_synch = key_stable_out) then
            delay_cntr := 0;
        else
            delay_cntr := delay_cntr + 1;
        end if;
        if (delay_cntr = delay_const) then
            key_stable_out <= key_synch;
            delay_cntr := 0;
        end if;
    end if;
end process;
```

27. Wymień i krótko scharakteryzuj trzy sposoby syntezy z wykorzystaniem bloków funkcjonalnych w układach FPGA.

- Synteza strukturalna** – z użyciem tablic przeszukiwania (Look-Up Tables, **LUT**) oraz makrokomórek logicznych.
- Synteza na poziomie transferu rejestrów (Register Transfer Level, RTL)** – z wykorzystaniem rejestrów oraz jednostek arytmetyczno-logicznych (**ALU**, Arithmetic Logic Unit).
- Synteza blokowa** – z wykorzystaniem gotowych bloków własności intelektualnej (**IP**, Intellectual Property), takich jak bloki cyfrowego przetwarzania sygnałów (**DSP**, Digital Signal Processing) oraz wbudowane pamięci.

(mug) W paczce takie coś napisali i to się bardziej z wykładem pokrywa:

Trzy sposoby syntezy:

- Primitives** - w syntezie automatycznie rozpoznawane są typowe bloki funkcjonalne i stosowane są znane przepisy na ich realizację,
- Inferred macros** - w syntezie rozpoznawane są w kodzie VHDL odpowiednio opisane komponenty (makra) jako bloki funkcjonalne i wykorzystywane są gotowe schematy takich bloków dla układu. Rozpoznawanie wymaga jednak od twórcy kodu świadomości istnienia takich bloków i stworzenie kodu w VHDL zgodnie z ich konstrukcją.
- Instantiated macros** - Jawnie wykorzystujemy blok funkcjonalny określając rodzaj modułu, jego konfigurację, oraz połączenia z sygnałami.

28. Podaj właściwości zmiennych w procesach.

- a. Widoczne tylko lokalnie,
- b. aktualizowane natychmiast (`:=`),
- c. zachowują wartość między aktywacjami procesu

29. Podaj właściwości zmiennych w funkcjach.

- a. Inicjowane przy każdym wywołaniu,
- b. nie zachowują wartości między wywołaniami,
- c. lokalne dla funkcji

30. Wymień zasadnicze różnice pomiędzy zmiennymi a sygnałami.

- a. **Zmienne:** natychmiastowa zmiana, lokalne, użyteczne w symulacji.
- b. **Sygnały:** zmiana po zakończeniu procesu, współdzielone, używane do opisu sprzętu

Cześć D

31. Omów sposoby parametryzacji modułów projektowych.

Do przypisywania modułom parametrów służy tzw. „generic”. Są one trochę analogiczne do templatów w C++.

Przykład:

```
entity niceone is
  generic (
    width: integer := 9;
  );
  port (
    ...
  );
end niceone;
```

Ten moduł ma parametr o nazwie „width” z domyślną wartością równą 9.

Wartość domyślną można również zmienić w momencie deklaracji komponentu:

```
component niceone is
  generic (
    width: integer := 30;
  );
  port (
    ...
  );
end component niceone;
```

Podobnie jak używamy „port map” do przypisania sygnałów, używamy „generic map” do przypisania parametrów w momencie instancjacji komponentu:

```
Instancja: niceone
GENERIC MAP(
  width => 8
)
PORT MAP(
  ...
```

);

W ten sposób jeśli np. używamy 5 takich samych komponentów, to łatwo można nadać wszystkim ten sam parametr, albo każdemu różny.

32. Co to jest „unconstrained vector”, podaj zastosowania.

Tworząc std_logic_vector możemy podać jego rozmiar (np. (7 downto 0)), ale nie musimy. W takim wypadku syntezytor utworzy wektor o takim rozmiarze, jaki będzie potrzebny.

33. Co to są cykle delta, podaj przykład.

Jest to krok czasu w symulacji, o długości 0.

Na przykład, jeśli mamy proces:

```
example: process () begin
    a <= not a;
    b <= a;
    c <= b;
    wait for 5ns;
end process example;
```

(zakładamy, że wartość początkowa a to 0)

to będą miały miejsce 3 zdarzenia (przypisanie a, przypisanie b i przypisanie c) które trzeba będzie symulować.

Jednak w symulacji, wartości a, b i c zostaną ustawiona na 1 w chwili 0.00. Dzieje się tak, ponieważ symulator przyjmuje 0 za czas trwania tych zdarzeń.

34. Jaka musi być liczba iteracji syntezywalnej pętli ?

Znana w chwili syntezy (statyczna)

35. Do czego służy instrukcja „generate” ?

Służy do generowania połączeń w części współbieżnej architektury (czyli nie w procesie). Ma postać pętli. Przykładowo, jeśli mamy wektory a i b, i chcemy zrobić XOR na wszystkich elementach:

```
wymagana_etykieta: for i in 0 to result'size generate
    result(i) <= a(i) xor b(i);
end generate;
```

Można też robić bardziej skomplikowane operacje, na przykład powiedzmy że chcemy aby pierwsze 3 elementy były potraktowane bramką AND, a nie XOR. Wtedy zagnieżdżamy generatory:

```
wymagana_etykieta1: for i in 0 to result'size generate
    etykieta1: if i < 3 generate
        result(i) <= a(i) and b(i);
    end generate;
    etykieta2: if i > 2 generate
        result(i) <= a(i) xor b(i);
    end generate;
end generate;
```

Każde generate musi mieć etykietę, i nie można stosować else

36. Napisz proces synchroniczny w języku VHDL realizujący syntezywalny mnożnik n-bitowy (na liczbach binarnych bez znaku). Nie wykorzystujemy operatora * Proces powinien rozpoczynać proces mnożenia z chwilą wystąpienia sygnału mul_start (typu std_logic). W trakcie trwania

mnożenia proces wystawia sygnał `mul_busy` (typu `std_logic`). Mnożna i mnożnik podane na sygnały: `mul_in_1` oraz `mul_in_2` (są to sygnały typu `std_logic_vector`), a wynik powinien być podany na rejestr `mul_result` (2n bitowy – również `std_logic_vector`). Podać definicje używanych sygnałów.

Realizujemy mnożenie pisemne. Przykład: mnożymy 1101 i 0101 (13 i 5):

```
  1101
  0101
  ----
  1101
 00000
110100
0000000
  ----
1000001
```

Czyli w każdym cyklu zegara:

1. dodajemy liczbę 1 do wyniku, jeśli najmłodszy bit liczby 2 to 1
2. bit-shift liczby 1 w lewo
3. bit-shift liczby 2 w prawo

Do tego trzeba liczyć ile iteracji wykonaliśmy, żeby wiedzieć kiedy zakończyć.

```
SIGNAL in_1: STD_LOGIC_VECTOR(n-1 downto 0);
SIGNAL in_2: STD_LOGIC_VECTOR(2*n-1 downto 0);
```

–Akumulator

```
SIGNAL out: STD_LOGIC_VECTOR(2*n - 1 downto 0) := (others => '0');
```

–Ile bitów z `in_2` zostało przetworzonych

```
SIGNAL digits: integer := '0';
```

mnożnik: process (clk, rst)

if rst = '1' then

in_1(n-1 downto 0 => mul_in_1, others => '0');

in_2 <= mul_in_2;

out <= (others=>'0');

digits <= '0';

mul_busy <= '0';

mul_result <= (others=> '0');

elsif rising_edge(clk) then

if mul_start = '1' then

in_1(n-1 downto 0 => mul_in_1, others => '0');

in_2 <= mul_in_2;

out <= (others=>'0');

digits <= '0';

mul_busy <= '1';

elsif mul_busy = '1' then

if in_2(0) = '1' then

out <= std_logic_vector(unsigned(in_1) + unsigned(out));

end if;

– Shifting left

in_1 <= in_1(2n-2 downto 0) & '0';

– Shifting right

in_2 <= '0' & (n-1 downto 1);

if (digits + 1) = n then

mul_busy <= '0';

mul_result <= out;

```

        end if;
        digits <= digits+1;
    end if;
end if;
end process;

```

37. Napisz proces synchroniczny w języku VHDL realizujący syntezywalny n-bitowy licznik Johnsona. Proszę operować na sygnałach typu std_logic i std_logic_vector. Podać definicje używanych sygnałów. Zabezpieczyć licznik przed blokadą w stanie (lub stanach) zabronionych.

Licznik Johnsona wygląda tak:

Johnson counter				
State	Q0	Q1	Q2	Q3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
0	0	0	0	0

robimy „roll-right”, tylko że zamiast brać najmłodszy bit i dawać go jako najstarszy, to bierzemy jego negację.

– Aktualny stan licznika

SIGNAL stan: STD_LOGIC_VECTOR(n-1 downto 0) := (others => '0');

```

johnson: process (clk, rst) begin
    if rst = '1' then
        stan <= (others => '0');
    elsif rising_edge(clk) then
        stan <= (not stan(0)) & stan(n-2 downto 1);
    end if;
end process;

```

(mug) tu raczej powinno być

stan <= (not stan(0)) & stan(n-1 downto 1)

ale wtedy liczy w inną stronę niż na rysunku (zakładam że Q0 to bit 0) więc ostatecznie najlepiej

stan <= stan(n-2 downto 0) & (not stan(n-1))

38. Napisz proces synchroniczny w języku VHDL realizujący syntezywalną maszynę stanów sterującą pracą wycieraczek samochodowych. Maszyna dysponuje sygnałami wejściowymi: wipe_on (oznacza włączenie wycieraczek), home_pos (pozycja spoczynkowa wycieraczek),

end_pos (pozycja końcowa wycieraczek). Maszyna stanów generuje sygnały wyjściowe: motor_on (włącza silnik wycieraczek), motor_rev (przełącza kierunek obrotów na wsteczny). Proszę pamiętać o synchronizujących przerzutnikach D dla sygnałów wejściowych z czujników. Podać definicje używanych sygnałów.

O co chodzi z tą synchronizacją? Każdy sygnał wejściowy asynchroniczny puszczaemy przez 2 przerzutniki, żeby nie zablokował się w stanie nieustalonym.

```
SIGNAL flop1_home_pos: STD_LOGIC;  
SIGNAL sync_home_pos: STD_LOGIC;
```

```
SIGNAL flop1_wipe_on: STD_LOGIC;  
SIGNAL sync_wipe_on: STD_LOGIC;
```

```
SIGNAL flop1_end_pos: STD_LOGIC;  
SIGNAL sync_end_pos: STD_LOGIC;
```

```
type WiperState is (stop, up, down)
```

```
SIGNAL state: WiperState;
```

```
wiper: process (clk, rst) begin  
    if rising_edge(clk) then  
        flop1_home_pos <= home_pos;  
        sync_home_pos <= flop1_home_pos;  
        flop1_wipe_on <= wipe_on;  
        sync_wipe_on <= flop1_wipe_on;  
        flop1_end_pos <= wipe_on;  
        sync_end_pos <= flop1_end_pos;  
  
        if sync_wipe_on = '1' then  
            if sync_end_pos = '1' then  
                --Jesli jestesmy u gory to jedziemy na dol  
                state <= down;  
            elsif sync_home_pos = '1' then  
                --Jesli jestesmy na dole to jedziemy do gory  
                state <= up;  
            end if  
        else  
            if sync_home_pos = '1' then  
                state <= stop;  
            else  
                state <= down;  
            end if;  
        end if;  
  
        --Ustawiamy sygnały sterujące w zaleznosci od stanu:  
        if state=up then  
            motor_on <= '1';  
            motor_rev <= '0';  
        elsif state = down then  
            motor_on <= '1';  
            motor_rev <= '1';  
        else  
            --state = stop  
        end if;  
    end if;  
end process;
```

```
        motor_on <= '0';  
        motor_rev <= '0';  
    end if;  
end if;  
end process;
```